IQRA UNIVERSITY IU

AI Lab # 6

This lab explores optimization-based search strategies by implementing Hill Climbing and Genetic Algorithms to iteratively improve solutions in state-space problems.

Artificial Intelligence (LAB)

Instructor: Mubbashir Ahmed

Email: mubbashir.ahmed@iqra.edu.pk

Date: 15 August, 2026

Today's Agenda

IQRA UNIVERSITY IU

01 What is Hill Climbing?

02 Algorithm Steps and Example

03 What is Genetic Algorithms?

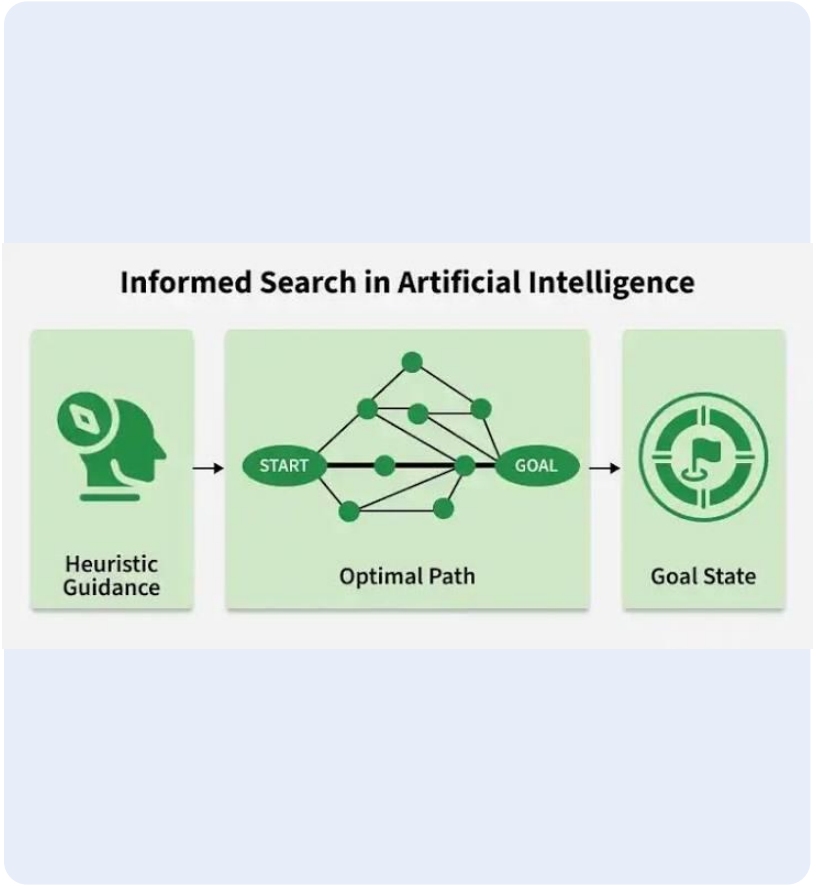
04 Algorithm Steps and Example

05 Lab Tasks

Informed Search

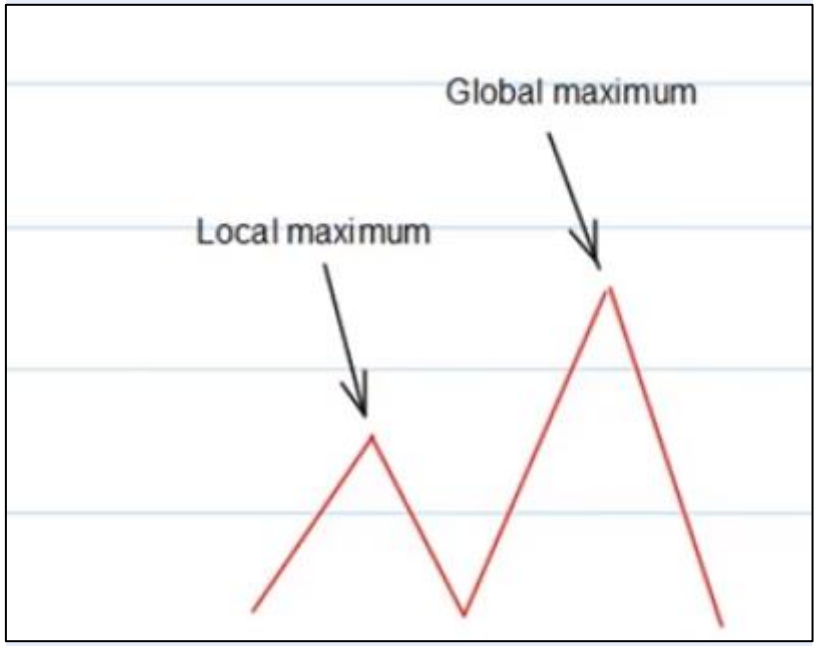
- Informed search (also called ***heuristic search***) uses **extra information** (called a ***heuristic***) to find solutions faster and smarter than uninformed methods like BFS, DFS, or UCS.
- A heuristic is like a guess or clue that tells the algorithm how close you are to the goal.

Imagine you're playing hide and seek, and someone tells you: "I'm here".



What is Hill Climbing?

- A simple optimization algorithm that starts with a random solution and then repeatedly moves to a “better” solution.
- Finds quick solution, works well for certain problems like TSP, traveling salesman problem.
- Finds local optimal solution not global optimal. Example: Finds ***“good” answer from the paper***, but not the ***“best” answer from the book***.

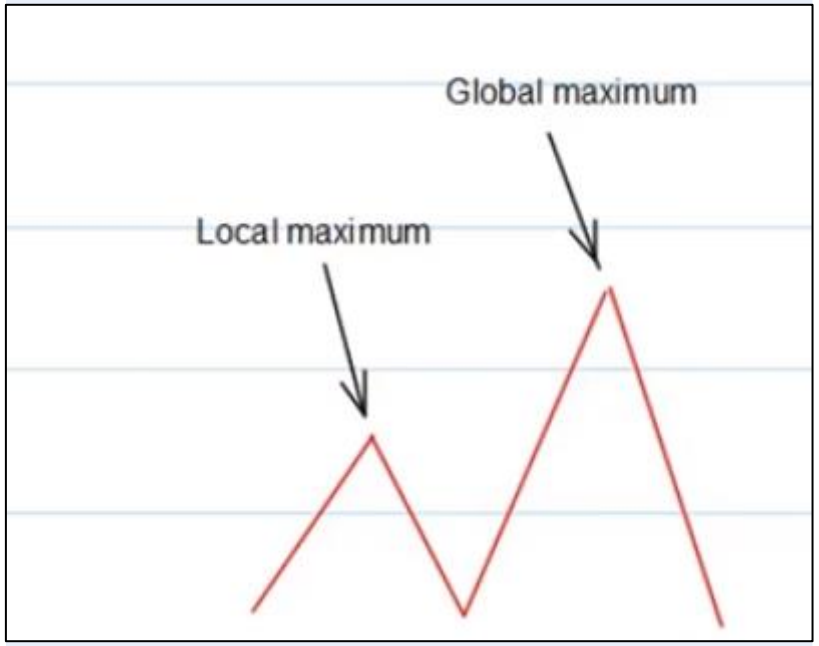


Imagine climbing a hill in fog:

- You cannot see the whole mountain.
- You only feel which direction is going upward.
- You take steps toward higher ground.
- You stop when every direction seems lower.

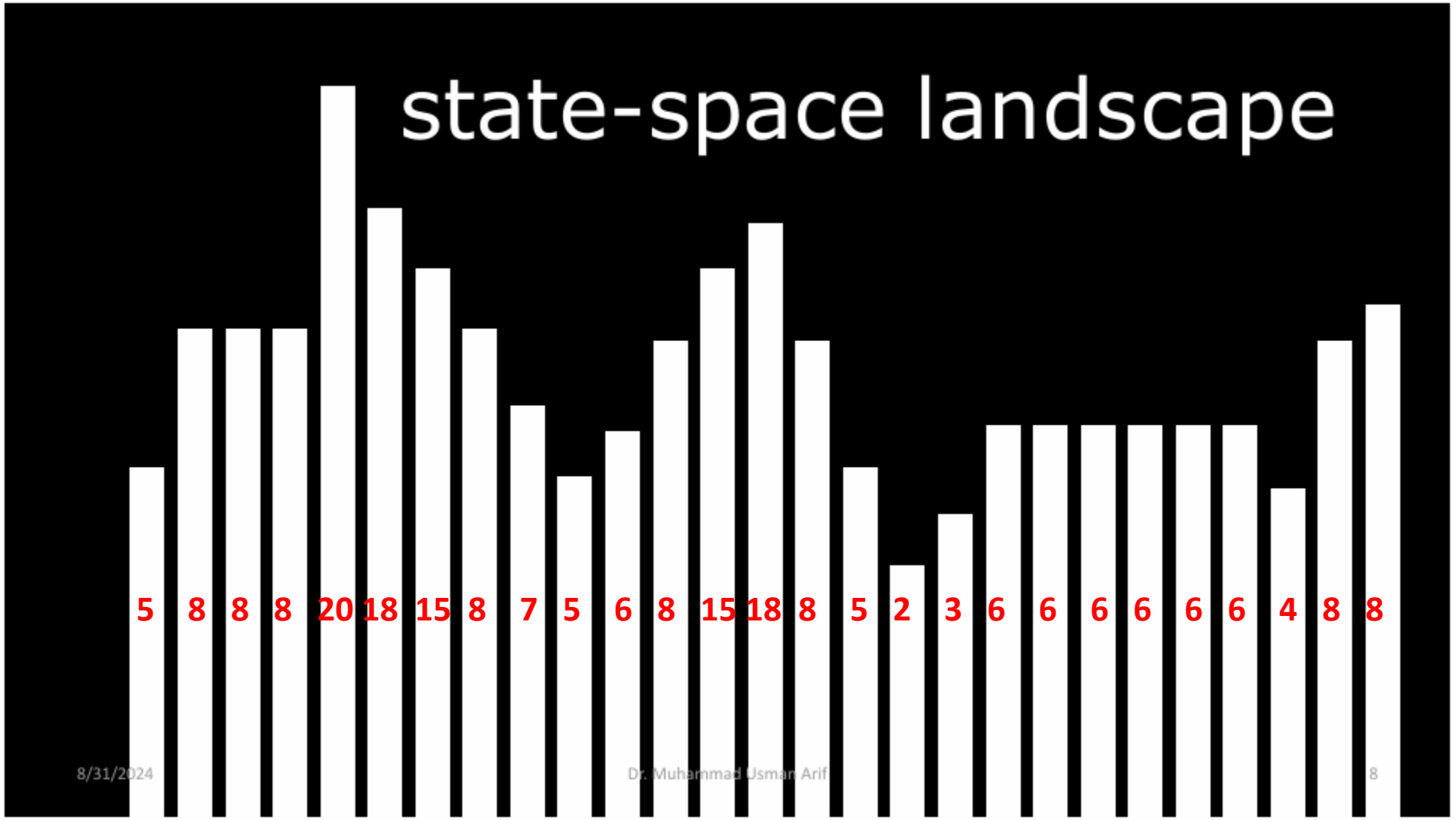
Applications of Hill Climbing

- GPS Navigation – to quickly reach a destination using shortest-looking turns.
- Robot pathfinding – to move toward a goal using nearest steps.
- Internet routing – selecting fastest-looking paths for data packets.
- Job scheduling – picking the task that looks quickest to finish.
- Machine Learning (ML) Optimization – Used for hyperparameter tuning to maximize the model's accuracy score.
- Traveling Salesman Problem (TSP) – Finding a near-optimal, short route by locally swapping the order of cities.
- Game AI – Evaluating and choosing the locally best next move in complex board games (e.g., Chess) to achieve a high heuristic score.
- Neural Network Weight Adjustment – Fine-tuning network weights by making small changes to minimize the training error function



Algorithm Steps

1. **Start with an initial solution:** Begin at a random or given starting node (current state)
2. **Evaluate neighbors:** Look at all neighboring nodes and calculate their heuristic value h(n)
3. **Pick the best neighbor:** Select the neighbor with the best heuristic value
4. **Compare with current node:** If the best neighbor is better than the current node, move to it. If not, stop — you've reached a peak (local or global optimum)
5. **Repeat until no improvement is possible:** Keep repeating steps 2 to 4 until every neighbor is worse than (or equal to) the current node, then return the final solution


state-space landscape
5 8 8 8 20 18 15 8 7 5 6 8 15 18 8 5 2 3 6 6 6 6 6 6 4 8 8

IQRA UNIVERSITY IU

Example of Hill Climbing

Now that we've covered Hill Climbing, let’s move towards practical code-based example

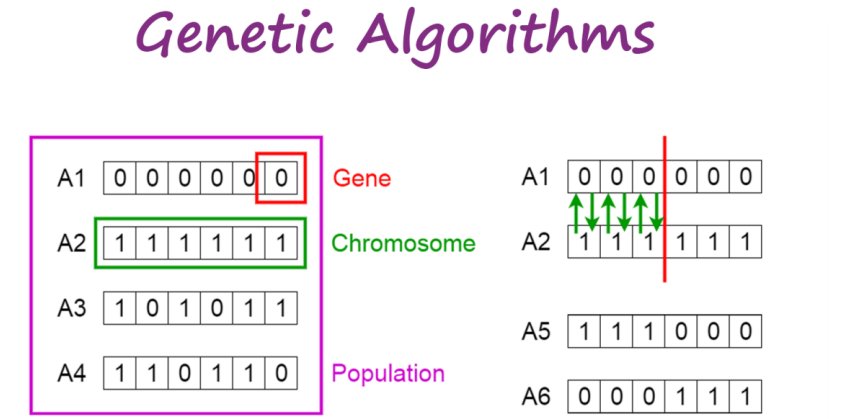
Hill Climbing

Hill Climbing on nodes A, B, C, D, E, F, G, H, I, J, K, L, M, N, O. Starting node is S and Goal node is M.

```
graph = { 'S': ['A', 'B'], 'A': ['C', 'D', 'E'], 'B': ['F', 'G'], 'C': ['H', 'I'], 'G': ['J', 'K'], 'I': ['L', 'M'], 'K': ['N', 'O’] }
heuristic = { 'S': 12, 'A': 9, 'B': 11, 'C': 8, 'D': 9, 'E': 7, 'F': 9, 'G': 9, 'H': 6, 'I': 5, 'J': 7, 'K': 6, 'L': 2, 'M': 0, 'N': 4, 'O’: 4 }

def hill_climbing(start):
    current = start
    path = [current]
    while True:
        neighbors = graph.get(current, [])
        if not neighbors:
            break

        best = neighbors[0]
        best_value = heuristic[best]
        for n in neighbors:
            if heuristic[n] < best_value:
                best_value = heuristic[n]
                best = n
        if heuristic[best] >= heuristic[current]:
            break
        current = best
        path.append(current)

    print("Hill-Climbing Path without back tracking:", path)

hill_climbing('S')
```

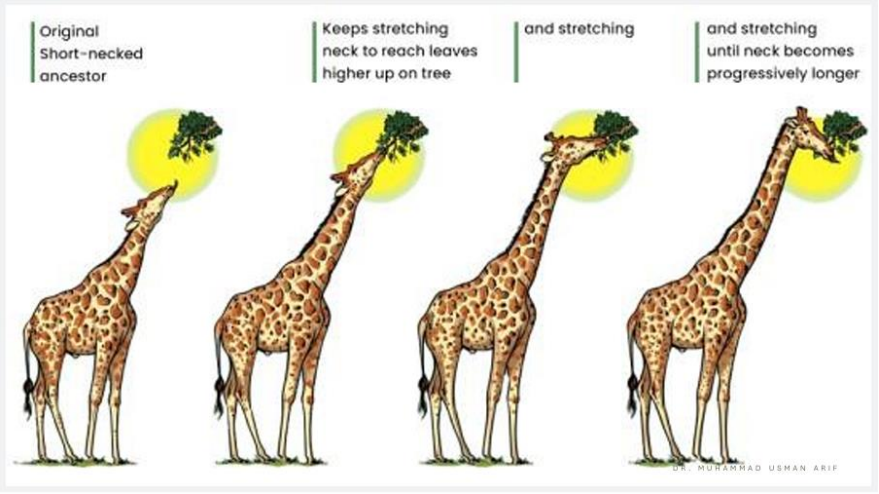
What is Genetic Algorithms?

- Concept: “Can computers solve problems the same way nature solves them?”
- Genetic Algorithms borrow the language of biological evolution: genes, chromosomes, survival of the fittest, and generations.
- Over many generations, solutions evolve toward the best one.



Core Concepts:

- **Population:** Start with possible solutions, like creatures in nature.
- **DNA:** Each solution carries parameters that describe it.
- **Crossover:** The best solution survives and combine.
- **Mutation:** A random change explores a new possibility.



Algorithm Steps

1. **Initialize population:** Start with a set of random solutions (e.g. 4 random numbers between 0 and 31), each treated as an individual's "DNA"

2. **Calculate fitness and Selection:** Evaluate each individual using a fitness function to measure how good the solution is (e.g., fitness = the number itself, bigger is better). Sort the population and pick the top individuals (parents) with the best fitness values

3. **Crossover:** Combine parts of the two parents' binary representations at a random or chosen crossover point to produce a new child

4. **Mutation:** With a small probability (e.g., 30%), randomly flip a bit in the child to introduce variation and explore new possibilities

5. **Replacement:** Replace the worst individual in the population with the new child

6. **Repeat until convergence:** Repeat steps 2 to 6 for several generations until the optimal solution is found or a fixed number of generations is reached

Example Problem: We want to find the largest number between 0 and 31

- **Population**: Start with 4 random numbers. **[5, 12, 7, 20]**
- **Fitness:** The bigger the number → the better.
- **Selection**: Pick the top 2 numbers. **20 and 12**
- **Crossover:** Mix digits (binary) of the two numbers to create a new number.
  - **20 = 10 | 100**
  - **12 = 01 | 100**
  - **20: 10100 → child**
- **Mutation:** Randomly change one bit to explore new numbers. **10100 → 10101 (21)**
- **Replace worst number** in population with new child. Take the new number (child) and replace the smallest number in the list.
  [5, 12, 7, 20] → child = 21 → replace 5 → [21, 12, 7, 20]

Tips

- In Python the **| is a bitwise OR operator,** but when we use it like this in crossover, its effect **acts like "combining bits"**.
- **There is no strict rule:** the **crossover** point can be **fixed or random**.

IQRA UNIVERSITY IU

Example of Genetic Algorithms

Now that we've covered Genetic Algorithms, let’s move towards practical code-based example

Genetic Algorithms

Genetic Algorithms for random population for size of 32.

```
import random

population = [random.randint(0, 31) for _ in range(4)]
print("Initial population:", population)

for generation in range(10):
    population.sort(reverse=True)  # bigger = better
    parent1, parent2 = population[0], population[1]
    child = (parent1 & 0b11100) | (parent2 & 0b00011)  # simple mix
    if random.random() < 0.3:
        child ^= 1
    population[-1] = child
    print(f"Gen {generation+1}: {population}")
    if 31 in population:
        print("Solution found!")
        break
```

ACTIVITY

Lab Task: Hill Climbing & Genetic Algorithms (Informed Search)

- *Implement the Hill Climbing algorithm to solve the 8-queens problem and display the number of attacking pairs at each move until a local optimum is reached.*
- *Write a simple Genetic Algorithm that tries to maximize a fitness function of your choice (such as number of 1s in a 5-bit binary string). Show population updates for at least 5 generations.*

Thank You

Questions or discussions are welcomed!